SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Beffy A Shanika	**Reg. No.:**	192511387
Date:	21/10/2026	**Duration:**	60 minutes

Code of Conduct

I Beffy A Shanika (192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Beffy*

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description
   - Scope of the System

- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

ANSWERS

Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform

1. Study and Analyze the Scenario

1.1 Business Domain

The proposed system belongs to the software engineering, application modernization, DevOps, and software maintenance domain. It is designed to help organizations transform traditional legacy applications into modern, independent microservices without interrupting their existing business operations.

1.2 Stakeholders

Stakeholder	**Role**
System Administrator	Manages users, access permissions, and system configuration.
Software Developer	Analyses legacy code and refactors selected components.
Software Architect	Identifies tightly coupled modules and designs microservice architecture.
DevOps Engineer	Manages containers, CI/CD pipelines, deployments, and rollback operations.
QA/Test Engineer	Creates and executes regression and integration tests.
Project Manager	Monitors migration progress, risks, and project reports.
Operations/Monitoring Team	Monitors application health, performance, and failures.
Organization/Client	Provides legacy applications and evaluates modernization outcomes.

1.3 Existing Challenges

1. Legacy applications contain tightly coupled modules, making modifications difficult.
2. Technical debt increases maintenance costs and development time.
3. Obsolete dependencies may cause security vulnerabilities and compatibility problems.
4. Complete system replacement can result in downtime and business disruption.
5. Manual testing may fail to identify regression errors after migration.
6. Deployment without proper automation increases the risk of production failures.
7. Limited monitoring makes it difficult to identify performance issues.
8. Lack of rollback mechanisms makes failed migrations difficult to recover from.
9. Poor documentation makes legacy system behaviour difficult to understand.

1.4 System Objectives

The main objective is to develop a platform that analyses legacy applications and supports their gradual modernization into microservices.

The specific objectives are:

- Identify technical debt, obsolete dependencies, and tightly coupled components.
- Determine which legacy modules are suitable for migration.
- Support API-based communication between legacy systems and microservices.
- Automate regression testing and integration testing.
- Support containerization, version control, and CI/CD.
- Monitor application performance and migration failures.
- Provide rollback and recovery mechanisms.
- Generate reports showing improvements in maintainability, reliability, scalability, and deployment performance.

2. Problem Description

Many organizations depend on legacy software that has been developed over several years. Such applications often contain outdated technologies, complex dependencies, duplicated code, and tightly connected modules. These limitations make the software difficult to maintain, test, scale, and integrate with modern applications.

Replacing the entire legacy system at once is expensive and risky because important business processes may depend on its existing behaviour. Therefore, organizations require a platform that supports gradual modernization while keeping existing services operational.

The proposed Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform will analyse legacy source code, identify high-risk components, recommend migration priorities, and support the conversion of selected modules into independent microservices. It will integrate automated testing, containerization, CI/CD pipelines, monitoring, and rollback facilities.

The system will also generate measurable reports to help organizations evaluate whether modernization has reduced technical debt and improved overall software quality.

3. Scope of the System

3.1 In-Scope Features

- Legacy source-code and dependency analysis.
- Technical-debt identification and reporting.
- Coupling and migration-risk assessment.
- Component prioritization for incremental migration.
- API design and microservice migration tracking.
- Container image and configuration management.
- Version control integration.
- Automated regression and integration testing.
- CI/CD pipeline integration.
- Application health and performance monitoring.
- Deployment failure detection and rollback support.
- Documentation and modernization progress reports.

3.2 Out-of-Scope Features

- Replacing every legacy application automatically without human review.
- Guaranteeing that all business logic can be converted without changes.
- Managing unrelated business operations such as payroll or inventory.
- Automatically modifying production systems without authorization.
- Eliminating every technical-debt issue in a single migration.

3.3 Proposed System Workflow

1. Register Legacy Application

Upload or connect source code and dependencies

2. Analyse and Assess

Identify technical debt, coupling, obsolete dependencies, and risks

3. Plan Incremental Migration

Select components and define API boundaries

4. Refactor and Containerize

Develop microservices and prepare container configurations

5. Test and Deploy

Run regression tests and CI/CD validation before release

6. Monitor and Recover

Track service health, compare results, and roll back failed deployments

The workflow follows an incremental modernization approach. Each selected component is analysed, migrated, tested, and monitored before the next component is migrated.

4. Functional Requirements

Functional requirements describe the operations the system must perform.

ID	**Functional requirement**	**Description**
FR-01	User Authentication	The system shall allow authorized users to log in and log out securely.
FR-02	Project Management	The system shall allow users to create projects and register legacy applications.
FR-03	Source Code Analysis	The system shall analyse source code and identify complex or tightly coupled modules.
FR-04	Technical-Debt Detection	The system shall identify code duplication, maintainability issues, and other technical-debt indicators.
FR-05	Dependency Analysis	The system shall identify outdated libraries and potentially vulnerable dependencies.
FR-06	Risk Assessment	The system shall calculate or classify component migration risks based on defined criteria.
FR-07	Migration Planning	The system shall help users prioritize components for incremental modernization.
FR-08	API Management	The system shall support documenting and managing API interfaces between legacy applications and microservices.
FR-09	Code Refactoring Support	The system shall support developers in modifying selected components for migration.
FR-10	Containerization	The system shall support container configurations for migrated microservices.
FR-11	Version Control	The system shall integrate with version control systems to track source-code changes.
FR-12	Automated Testing	The system shall execute configured regression and integration tests.

FR-13	CI/CD Integration	The system shall support automated build, test, and deployment pipelines.
FR-14	Deployment Management	The system shall support deploying migrated services to configured environments.
FR-15	Rollback Management	The system shall support restoring a previous stable deployment when a failure occurs or rollback is authorized.
FR-16	Performance Monitoring	The system shall collect service health, response-time, error-rate, and resource-usage metrics where supported.
FR-17	Failure Alerts	The system shall notify authorized users about test failures, deployment errors, and service-health problems.
FR-18	Progress Reporting	The system shall generate reports showing migration status, identified technical debt, test results, and performance comparisons.
FR-19	Documentation Management	The system shall store migration plans, API documentation, test results, and maintenance records.
FR-20	Access Control	The system shall restrict project operations according to user roles and permissions.

5. Non-Functional Requirements

Non-functional requirements describe the quality attributes and operating conditions of the platform.

ID	Requirement	Description
NFR-01	Performance	Code analysis and report generation shall complete within agreed time limits for supported project sizes.
NFR-02	Security	The platform shall protect source code, credentials, configuration files, and project information against unauthorized access.
NFR-03	Reliability	The platform shall execute migration workflows consistently and record errors for recovery.
NFR-04	Availability	The platform shall target at least 99.5% monthly availability, excluding agreed maintenance periods, if supported by the deployment environment.
NFR-05	Scalability	The platform shall support increasing numbers of projects, users, and microservices through configurable resource expansion.
NFR-06	Maintainability	The platform shall use modular architecture, documented interfaces, and maintainable code.
NFR-07	Usability	The interface shall provide understandable dashboards, reports, and migration status indicators.
NFR-08	Compatibility	The platform shall support the source languages, repositories, container technologies, and CI/CD tools selected for the project.

NFR-09	Recoverability	The platform shall preserve deployment history and support recovery to a known stable version.
NFR-10	Auditability	The platform shall maintain logs of important user actions, configuration changes, deployments, and rollbacks.
NFR-11	Interoperability	The platform shall communicate with external tools through supported APIs and interfaces.
NFR-12	Accuracy	Analysis results and modernization metrics shall be calculated using documented rules and clearly stated limitations.

Note: The numerical availability target is a proposed requirement. Actual performance and availability targets should be finalized after stakeholder discussion and testing.

6. Identify Actors and Use Cases

6.1 Primary Actors

- System Administrator: Manages users, roles, and system configuration.
- Software Developer: Analyses and refactors legacy code and develops migrated services.
- Software Architect: Reviews component dependencies and approves migration plans.
- DevOps Engineer: Configures containers, pipelines, deployments, and rollbacks.
- QA/Test Engineer: Manages test cases and reviews regression-test results.
- Project Manager: Reviews modernization progress, risks, and reports.

6.2 Secondary Actors

- Version Control System: Stores source code and tracks changes.
- Static Analysis/Dependency Scanner: Provides code-quality and dependency analysis.
- CI/CD Platform: Builds, tests, and deploys software.
- Container Runtime or Orchestrator: Runs containerized services.
- Monitoring and Alerting System: Collects metrics and sends alerts.
- API Gateway or API Management Tool: Supports routing and management of service interfaces.

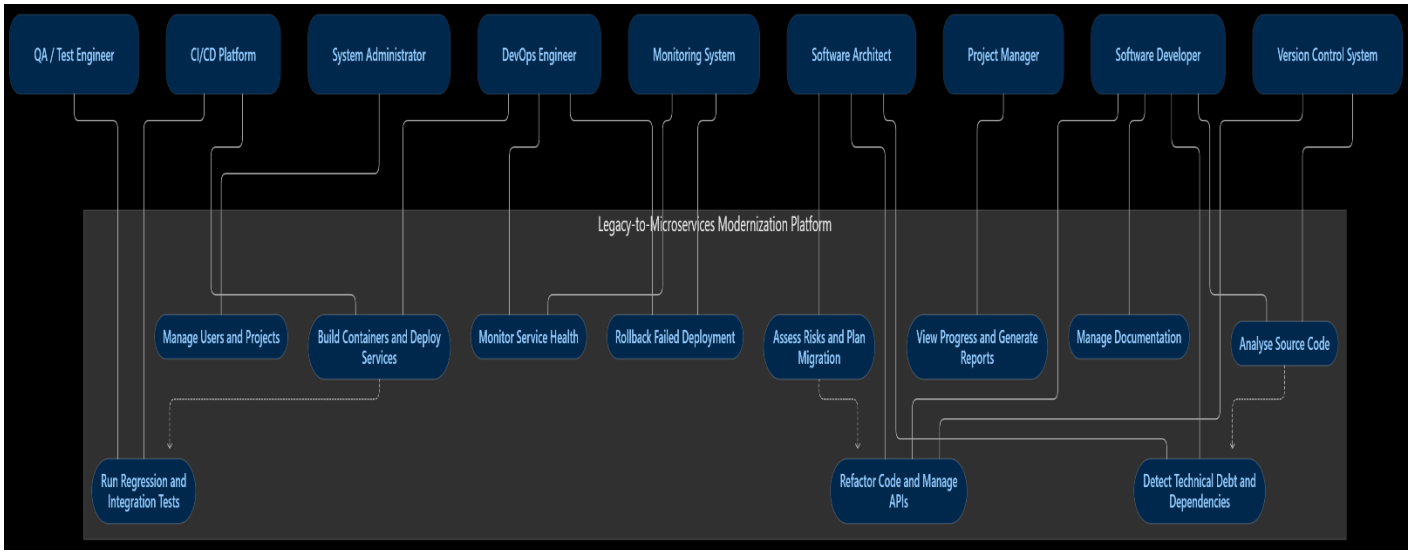
6.3 Major Use Cases

Actor	**Use cases**
System Administrator	Log in, manage users, assign roles, configure integrations
Software Developer	Register project, analyse code, review technical debt, refactor code, update documentation
Software Architect	Review dependencies, assess migration risks, prioritize components, design APIs, approve migration plans

DevOps Engineer	Configure containers, manage CI/CD, deploy services, monitor deployments, initiate rollback
QA/Test Engineer	Create test plans, run regression tests, execute integration tests, review test results
Project Manager	View dashboards, track migration progress, compare metrics, generate reports
Version Control System	Retrieve source code, record changes, provide version history
CI/CD and Monitoring Tools	Execute pipelines, report deployment status, provide service metrics and alerts

6.4 Use Case Diagram

The following diagram represents the main interactions between the actors and the modernization platform.



7. Use Case Descriptions

Use Case 1: Analyse Legacy Application

Field	Description
Use Case ID	UC-01
Primary Actor	Software Developer
Objective	Identify technical debt, complex modules, and dependency problems.
Preconditions	The user is authenticated and has permission to access the project.
Main Flow	1. The developer selects a project. 2. The developer starts source-code analysis. 3. The system scans supported source files and dependencies. 4. The system identifies code-

	quality issues and tightly coupled components. 5. The results are saved and displayed in a report.
Alternative Flow	If the project contains unsupported files or incomplete dependencies, the system reports the limitation and displays the available results.
Postconditions	Analysis results are stored and available for migration planning.

Use Case 2: Plan Incremental Migration

Field	**Description**
Use Case ID	UC-02
Primary Actor	Software Architect
Objective	Select suitable components for gradual modernization.
Preconditions	Source-code analysis has been completed.
Main Flow	1. The architect reviews component dependencies and risk reports. 2. The system displays candidate components. 3. The architect evaluates business criticality, complexity, and dependencies. 4. The architect selects components and defines migration priorities. 5. The system saves the migration plan.
Alternative Flow	If a component has excessive dependencies or unacceptable risk, it is deferred for further analysis or refactoring.
Postconditions	An approved or draft migration plan is available for implementation.

Use Case 3: Test and Deploy a Microservice

Field	**Description**
Use Case ID	UC-03
Primary Actor	DevOps Engineer
Supporting Actor	QA/Test Engineer
Objective	Validate and deploy a migrated microservice.
Preconditions	A migrated component, its configuration, and required test cases are available.
Main Flow	1. The developer commits the changes. 2. The CI/CD pipeline builds the service. 3. Automated regression and integration tests are executed. 4. If the tests pass, the pipeline creates or verifies the container image. 5. The service is deployed to the configured environment after required approvals. 6. Deployment status is recorded.
Alternative Flow	If the build or tests fail, deployment is stopped and the failure is reported. If deployment causes a critical issue, an authorized rollback can be initiated.
Postconditions	The service is deployed successfully, or the failure and recovery status are recorded.

Use Case 4: Monitor and Roll Back a Deployment

Field	Description
Use Case ID	UC-04
Primary Actor	DevOps Engineer
Supporting Actor	Monitoring System
Objective	Detect deployment problems and recover from failures.
Preconditions	A service has been deployed and monitoring is configured.
Main Flow	1. The monitoring system collects health and performance metrics. 2. The platform compares metrics against configured thresholds. 3. If an abnormal condition is detected, an alert is generated. 4. The DevOps engineer reviews the issue. 5. The engineer authorizes rollback if necessary. 6. The previous stable deployment is restored, and the result is recorded.
Alternative Flow	If rollback is unsuccessful, the platform raises a critical alert and provides diagnostic information for manual recovery.
Postconditions	The service is monitored, and any recovery action is recorded.

Use Case 5: Generate Modernization Report

Field	Description
Use Case ID	UC-05
Primary Actor	Project Manager
Objective	Measure modernization progress and software quality improvements.
Preconditions	Analysis, testing, deployment, or monitoring data is available.
Main Flow	1. The project manager selects a project and reporting period. 2. The system retrieves available metrics. 3. The system compares baseline and current measurements. 4. The system generates a report showing technical debt, migration status, test results, failures, and performance. 5. The report is displayed or exported in a supported format.
Alternative Flow	If baseline or current data is unavailable, the report identifies missing measurements instead of presenting unsupported comparisons.
Postconditions	A report is available for project evaluation and decision-making.

8. Data Requirements

The platform must store and manage the information required for code analysis, migration planning, testing, deployment, and monitoring.

8.1 Main Data Entities

Entity	Important attributes
User	User ID, name, email, role, authentication details
Project	Project ID, project name, description, repository URL, creation date
Legacy Application	Application ID, project ID, programming language, framework, version
Component	Component ID, name, source location, dependencies, complexity score
Technical Debt	Issue ID, component ID, issue type, severity, description, status
Migration Plan	Plan ID, component ID, priority, risk level, planned status, approval status
API Details	API ID, service name, endpoint, method, version, specification
Test Execution	Test ID, test type, execution date, pass/fail status, error details
Deployment	Deployment ID, service version, environment, status, deployment time
Rollback Record	Rollback ID, deployment ID, previous version, reason, status
Monitoring Metric	Metric ID, service ID, response time, error rate, resource usage, timestamp
Audit Log	Log ID, user ID, action, timestamp, outcome

8.2 Data Processing and Storage

1. Source-code analysis results shall be linked to the relevant project and component.
2. Every identified technical-debt issue shall have a unique identifier and severity level.
3. Migration plans shall maintain component priorities and approval status.
4. Test records shall include execution results and relevant failure details.
5. Deployment and rollback histories shall be retained for troubleshooting.
6. Monitoring records shall contain timestamps to support performance comparisons.
7. Access credentials and sensitive configuration values shall be stored securely.
8. The system shall apply suitable retention and backup policies to important records.

8.3 Data Quality Requirements

- Each project and component shall have a unique identifier.
- Required fields shall be validated before records are saved.
- Related records shall maintain referential integrity.
- Metrics shall include units, timestamps, and measurement definitions where applicable.
- Missing or incomplete information shall be identified clearly.
- Duplicate records shall be minimized through validation and unique identifiers.

9. External Interface Requirements

9.1 User Interface

The platform shall provide a web-based interface containing:

- Login and user management pages.
- Project registration and source-code connection pages.
- Technical-debt analysis dashboards.
- Component dependency and risk views.
- Migration planning and approval screens.
- Test execution and CI/CD status pages.
- Deployment and rollback controls.
- Monitoring dashboards and alert notifications.
- Progress reports and documentation pages.

The interface should use clear labels, readable charts, search options, and status indicators.

9.2 Software Interfaces

External system	Purpose
GitHub, GitLab, or similar	Source-code retrieval, commits, branches, and version history
Static analysis tools	Code-quality, complexity, and technical-debt analysis
Dependency scanners	Detection of outdated or vulnerable packages
API management tools	API specification, validation, and interface management
Docker or compatible container technology	Building and running containerized services
Jenkins, GitHub Actions, or similar	Automated build, test, and deployment pipelines
Monitoring tools	Collection of service health, logs, metrics, and alerts
Database management system	Storage of project data, analysis results, and migration history

These integrations are proposed options. The final implementation may select a smaller set depending on project scope and available resources.

9.3 Hardware Interface

- A computer or server shall host the platform.
- Developers shall access the system through a compatible computer or mobile browser where supported.
- Sufficient CPU, memory, and storage shall be available for source-code analysis and test execution.
- Additional resources may be required for large applications or multiple concurrent pipelines.

9.4 Communication Interface

- The platform shall use HTTPS for secure web communication.
- REST APIs or other supported interfaces shall be used for external integrations.
- Authentication tokens and service credentials shall not be exposed in logs or source code.
- Communication errors and integration failures shall be recorded and reported.

10. System Features

The proposed platform consists of the following major features.

Feature 1: Legacy Application Analysis

Scans supported source code and dependencies to identify tightly coupled components, complexity, duplicated code, and technical-debt indicators.

Feature 2: Migration Planning and API Design

Helps architects select suitable components, define service boundaries, plan API interfaces, and track migration progress.

Feature 3: Containerization and CI/CD

Supports container configurations, version control, automated builds, testing pipelines, and controlled deployments.

Feature 4: Automated Testing and Recovery

Executes regression and integration tests, records failures, and supports rollback to a previous stable deployment.

Feature 5: Monitoring and Reporting

Tracks service health, response time, errors, deployment status, technical debt, and modernization progress.

10.1 Measurable Success Criteria

To demonstrate the effectiveness of modernization, the platform shall compare baseline measurements with measurements collected after migration.

Metric	Measurement method	Expected improvement
Technical Debt	Count or severity of identified debt issues	Reduction
Code Complexity	Complexity scores for selected components	Reduction where appropriate
Test Pass Rate	Passed tests divided by total executed tests	Increase or maintenance of agreed quality level
Deployment Time	Time taken to complete a deployment	Reduction
Failure Rate	Failed deployments divided by total deployments	Reduction
Response Time	Average or percentile service response time	Improvement against the defined baseline
Service Availability	Available service time divided by total measured time	Increase or maintenance of target
Migration Progress	Completed components divided by planned components	Increase toward project completion

The expected improvements are targets to be verified through testing and real measurements, not guaranteed results. Comparisons should use consistent environments, measurement periods, and definitions.

11. Assumptions and Constraints

11.1 Assumptions

1. The organization has access to the source code or repositories of the legacy applications.
2. Users have the required permissions to analyse and modify the selected applications.
3. The source-code languages and frameworks are supported by the selected analysis tools.
4. Existing business requirements and important application behaviours can be documented or tested.
5. The organization provides suitable computing resources for analysis, testing, and deployment.
6. External tools such as version control, CI/CD, and monitoring systems provide the required integration interfaces.
7. Developers and architects review automated recommendations before critical migration decisions.
8. Baseline measurements are available or can be collected before migration.
9. The organization permits the use of containers and microservices in the selected environment.
10. Rollback is possible only when compatible previous versions, configurations, and required data-recovery procedures are available.

11.2 Constraints

Constraint type	Description
Technical	Some legacy applications may use unsupported languages, frameworks, or obsolete libraries.
Technical	Tight coupling and undocumented business rules may make component separation difficult.
Operational	Migration must not interrupt critical business functions beyond the organization's approved limits.
Operational	Deployment and rollback operations may require authorized approval and scheduled maintenance windows.
Economic	Limited budgets may restrict infrastructure, commercial tools, and testing resources.
Legal	Source code, credentials, customer information, and logs must be handled according to applicable laws and organizational policies.
Security	Access to repositories and production environments must be restricted to authorized users.
Time-related	The project must be completed within the academic or organizational deadline.
Resource-related	Limited team experience and hardware capacity may restrict the number of supported integrations.
Compatibility	The platform can analyse and migrate only the languages, tools, and environments supported by its implementation.

12. Validation of Requirement Completeness and Consistency

Requirement validation ensures that the SRS is complete, understandable, consistent, and suitable for implementation.

12.1 Completeness Checklist

Requirement validation checklist

0/10 reviewed

Legacy source-code analysis is included

Technical debt and obsolete dependencies are covered

Migration planning and risk assessment are included

API design and incremental modernization are covered

Regression and integration testing are included

Containerization and CI/CD are included

Monitoring, alerts, and rollback are covered

Security, access control, and audit logs are specified

Data storage and external interfaces are defined

Measurable success criteria and reporting are included

The checklist can be used during the final review of the SRS. Checking an item records that it has been reviewed; it does not independently prove that the requirement has been implemented or tested.

12.2 Ambiguity Validation

Some requirements need measurable acceptance criteria before development.

General requirement	Suggested improvement
The system should be fast.	Define response-time limits for analysis, dashboards, and report generation using agreed project sizes.
The system should be secure.	Define authentication, role permissions, encryption, and credential-management requirements.
The system should reduce technical debt.	Define baseline measurements and the calculation method used to measure change.
The system should be highly available.	Specify the availability target, maintenance exclusions, and measurement period.
The system should support rollback.	Define supported deployment types, approval rules, recovery conditions, and verification steps.

12.3 Redundancy Validation

- Technical-debt detection and source-code analysis are related but distinct: source analysis gathers information, while debt detection classifies issues.
- Monitoring and failure alerts are related but distinct: monitoring collects metrics, while alerting notifies users when defined conditions are met.
- Testing and deployment must remain separate steps because a successful build does not necessarily mean the software has passed its tests.
- Migration planning and migration execution must be distinguished to ensure that recommendations are reviewed before implementation.

12.4 Consistency Validation

The following consistency rules should be maintained:

1. A deployment shall not be marked successful until the configured deployment checks pass.
2. Failed mandatory tests shall prevent deployment unless an explicitly approved exception policy applies.
3. Rollback records shall refer to an existing deployment and a valid recovery target.
4. Migration reports shall use recorded measurements rather than assumed improvements.
5. User permissions shall be consistent across project management, deployment, and rollback operations.
6. The system shall clearly distinguish automated recommendations from approved migration decisions.

12.5 Suggested Improvements

- Define supported programming languages and frameworks for the first implementation.
- Specify the technical-debt scoring method and migration-risk calculation.
- Add role-based approval workflows for production deployments.
- Define backup and database migration procedures for services that share legacy data.
- Include API compatibility checks and security testing in the migration pipeline.
- Set concrete performance targets after identifying expected project sizes and available infrastructure.
- Test recovery procedures before using rollback mechanisms in production.

13. Conclusion

The proposed Legacy-to-Microservices Modernization and Technical-Debt Recovery Platform provides a structured approach to analysing and modernizing legacy applications. It supports the identification of technical debt, selection of migration candidates, API-based service separation, containerization, automated testing, CI/CD, monitoring, and rollback.

The platform emphasizes gradual migration instead of replacing the entire legacy system at once. This approach helps organizations control migration risks and maintain business continuity. Through measurable metrics and regular reporting, the organization can evaluate improvements in maintainability, scalability, reliability, and deployment performance.

Therefore, the proposed SRS establishes a clear foundation for designing, developing, testing, and evaluating the modernization platform.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25